

Venus Emergency Brake Feature Audit Report

Please Note

1. The analysis of the Severity is purely based on the smart contracts mentioned in the Audit Scope and does not include any other potential contracts deployed by the Owner. No applications or operations were reviewed for Severity. No product code has been reviewed.
2. Due to the time limit, the audit team did not do much in-depth research on the business logic of the project. It is more about discovering issues in the smart contracts themselves.

Audit Period: 2026/01/06 - 2026/01/09 (YYYY/MM/DD)

Overall Risk: **Medium**

Project Name	Venus Emergency Brake Feature Audit Report
Github	https://github.com/VenusProtocol/venus-periphery/pull/24

Smart contracts list:

No.	Contract Name	Verdict	Details
1	contracts/DeviationSentinel/DeviationSentinel.sol	Medium	[M01] [M02] [I01] [I02] [I03]
2	contracts/DeviationSentinel/Oracles/PancakeSwapOracle.sol	Informational	[I04]
3	contracts/DeviationSentinel/Oracles/SentinelOracle.sol	Informational	[I05]
4	contracts/DeviationSentinel/Oracles/UniswapOracle.sol	Informational	[I04]

Findings

[M01] Potential CF/ LT Overwrite When Restoring Core Pool Markets After `lastPoolId` Increases

Contract contracts/DeviationSentinel/DeviationSentinel.sol

Severity Level Medium

Description In `_setCollateralFactorToZero`, the contract stores the original collateral factors (poolCFs) and liquidation thresholds (poolLTs) for Core Pool markets using a mapping keyed by `poolId`. Later, `_restoreCollateralFactor` iterates from `corePoolId` to `lastPoolId` to restore CF and LT values.

```
JavaScript
function _setCollateralFactorToZero(IVToken market) internal {
    IComptroller comptroller = market.comptroller();
    MarketState storage state = marketStates[address(market)];

    if (address(comptroller) == address(CORE_POOL_COMPTRROLLER)) {
        // Store original CF and LT for each emode group, then set
        to 0
        for (uint96 i = CORE_POOL_COMPTRROLLER.corePoolId(); i <=
CORE_POOL_COMPTRROLLER.lastPoolId(); i++) {
            //...
        }
    } else {
        //...
    }
}

function _restoreCollateralFactor(IVToken market) internal {
    IComptroller comptroller = market.comptroller();
    MarketState storage state = marketStates[address(market)];

    if (address(comptroller) == address(CORE_POOL_COMPTRROLLER)) {
        // Core pool - restore original CF and LT for each emode
        group
        for (uint96 i = CORE_POOL_COMPTRROLLER.corePoolId(); i <=
CORE_POOL_COMPTRROLLER.lastPoolId(); i++) {
            //...
        }
    } else {
        //...
    }
}
```

If the Core Pool adds a new `poolId` after `_setCollateralFactorToZero` has been executed, the corresponding `state.poolCFs` and `state.poolLTs` entries for the new pool

will be uninitialized (default to 0). As a result, `_restoreCollateralFactor` may overwrite the collateral factor and liquidation threshold of newly added pools, unintentionally setting them to zero.

Impact:

- The CF and LT of the new pool are incorrectly set to 0, causing the asset to be unusable as collateral in the new pool.
- If a user has already used the asset as collateral in the new pool, its collateral value immediately becomes 0, significantly reducing borrowing capacity.
- An LT of 0 sets the liquidation threshold to 0, which may cause the account health factor to drop to 0, potentially triggering immediate liquidation.
- Users may suffer financial losses, and the protocol state becomes inconsistent, requiring manual intervention through governance to fix.

Recommendation Multi-layer protection

- Primary check: Compare stored values with current protocol state to distinguish new pools from original CF = 0 cases
 - New pool characteristic: storedCF = 0 (uninitialized) and currentCF != 0 (configured)
 - Original CF = 0 characteristic: storedCF = 0 (original value) and currentCF = 0 (currently also 0)
 - By comparing these two values, it is possible to accurately distinguish between new pools and original CF = 0 cases.
 - Implementation - Check Current Protocol State

JavaScript

```
function _restoreCollateralFactor(IVToken market) internal {
    IComptroller comptroller = market.comptroller();
    MarketState storage state = marketStates[address(market)];

    if (address(comptroller) == address(CORE_POOL_COMPTRROLLER))
    {
        for (uint96 i = CORE_POOL_COMPTRROLLER.corePoolId(); i <=
CORE_POOL_COMPTRROLLER.lastPoolId(); i++) {
            (bool isListed, uint256 currentCF, , uint256
currentLT, , , ) =
                CORE_POOL_COMPTRROLLER.poolMarkets(i,
address(market));

            if (isListed) {
                // Check: If stored value is 0 but current CF is
not 0,
                // this is a new pool added after
_setCollateralFactorToZero
                if (state.poolCFs[i] == 0 && currentCF != 0) {
                    continue; // Skip new pools
                }

                // If stored value is 0 and current CF is also
0,
```

```

// it might be the original value, allow
restoration

        uint256 result =
CORE_POOL_COMPTRROLLER.setCollateralFactor(
            i,
            address(market),
            state.poolCFs[i],
            state.poolLTs[i]
        );
        if (result != 0) revert
        ComptrollerError(result);

        emit CollateralFactorUpdated(address(market), i,
0, state.poolCFs[i]);
        delete state.poolCFs[i];
        delete state.poolLTs[i];
    }
}
}

```

- Defensive check: Validate restoration values for reasonableness
- Boundary validation: Ensure CF and LT are within reasonable bounds

```

JavaScript
if (isListed) {
    uint256 storedCF = state.poolCFs[i];
    uint256 storedLT = state.poolLTs[i];

    // Primary check: Skip new pools
    if (storedCF == 0 && currentCF != 0) {
        continue;
    }

    // Defensive check: Prevent LT = 0
    if (storedLT == 0) {
        continue; // Skip to prevent immediate liquidation
    }

    // Restore...
}

```

Status

Fixed in

<https://github.com/VenusProtocol/venus-periphery/pull/44/commits/390856be2fda7a0>

90e31a888372f6d58d3c933a0.

[M02] MMarket CF/LT May Be Incorrectly Modified

Contract contracts/DeviationSentinel/DeviationSentinel.sol

Severity Level **Medium**

Description

When a price deviation is detected, the DeviationSentinel contract temporarily sets a market's collateral factor (CF) to zero (while keeping the liquidation threshold, LT, unchanged) and caches the original CF/LT values in local storage, so that they can later be restored via `_restoreCollateralFactor` once the deviation is resolved.

However, within the Venus Protocol, market parameters such as CF, LT, and pause states can also be modified through other governance mechanisms outside of this contract. If such governance actions occur without promptly (or correctly) invoking the Sentinel's `resetMarketState` function to clear the cached state, the subsequent execution of `_restoreCollateralFactor` may unintentionally overwrite updated market parameters.

For example:

1. The Sentinel detects a price deviation for a given asset and sets the market's CF to 0, while caching the original CF/LT values in `marketStates`;
2. During the Sentinel's intervention, the market is delisted through governance;
3. The market is later re-listed, and new CF/LT parameters are configured via governance;
4. Once the Sentinel determines that the price deviation has been resolved, `_restoreCollateralFactor` is triggered;
5. Because the market is listed again, the Sentinel restores the previously cached (stale) CF/LT values, inadvertently overriding the parameters most recently set by governance.

In addition, within the `handleDeviation` function, if a price deviation occurs but the Sentinel's internal `marketStates` already indicates that the market is paused, the function returns early and no further pause or CF adjustment is performed. Consequently, if a market was originally paused by the Sentinel but later unpaused via another governance path without calling `resetMarketState`, the Sentinel's internal state may become inconsistent. This can cause subsequent price deviations for the same market to go undetected or unhandled, effectively disabling the intended price monitoring and risk mitigation logic.

JavaScript

```
if (hasDeviation) {  
  if (sentinelPrice > oraclePrice) {  
    // Early return if borrow is already paused
```

```

        if (state.borrowPaused) return; /// @audit If this
value is set to true, the contract will skip executing the pausing
logic or collateral factor adjustments.

        _pauseBorrow(market);
        state.borrowPaused = true;
    } else {
        // Early return if CF is already modified and supply
is already paused
        if (state.cfModifiedAndSupplyPaused) return;
        _setCollateralFactorToZero(market);
        _pauseSupply(market);
        state.cfModifiedAndSupplyPaused = true;
    }

```

Recommendation	The remediation can follow the approach described in M01. Specifically, during the restoration process, the contract should first verify that the cached values and the market's current state are reasonable before applying them. In addition, the project should ensure that whenever market parameters (such as CF/LT or pause states) are modified through other governance mechanisms, the <code>resetMarketState</code> function of this contract is called promptly to clear any cached state. This helps prevent outdated cached values from unintentionally overwriting the market's current parameters.
Status	Fixed in https://github.com/VenusProtocol/venus-periphery/pull/44/commits/390856be2fda7a090e31a888372f6d58d3c933a0.

[I01] Constructor Does Not Validate `corePoolComptroller_` For Zero

Contract	contracts/DeviationSentinel/DeviationSentinel.sol
Severity Level	Informational
Description	The constructor validates <code>resilientOracle_</code> and <code>sentinelOracle_</code> for zero addresses but does not validate <code>corePoolComptroller_</code>. If a zero address is passed, the contract will be deployed with an invalid <code>CORE_POOL_COMPROLLER</code>, causing all subsequent operations that use it to revert.

```

JavaScript
constructor(

```

```

    ICorePoolComptroller corePoolComptroller_,
    ResilientOracleInterface resilientOracle_,
    OracleInterface sentinelOracle_
) {
    // ⚠️ No zero address check
    CORE_POOL_COMPTRROLLER = corePoolComptroller_;
    if (address(resilientOracle_) == address(0)) revert ZeroAddress();
    // ✅ Checked
    if (address(sentinelOracle_) == address(0)) revert ZeroAddress();
    // ✅ Checked
    RESILIENT_ORACLE = resilientOracle_;
    SENTINEL_ORACLE = sentinelOracle_;
    // ...
}

```

Impact

If corePoolComptroller_ is address(0), the following operations will revert:

1. resetMarketState:

JavaScript

```

for (uint96 i = CORE_POOL_COMPTRROLLER.corePoolId(); ...)
// Revert: call to zero address

```

2. _setCollateralFactorToZero):

JavaScript

```

CORE_POOL_COMPTRROLLER.corePoolId() // Revert
CORE_POOL_COMPTRROLLER.poolMarkets(...) // Revert
CORE_POOL_COMPTRROLLER.setCollateralFactor(...) // Revert

```

3. _restoreCollateralFactor:

JavaScript

```

CORE_POOL_COMPTRROLLER.corePoolId() // Revert
CORE_POOL_COMPTRROLLER.poolMarkets(...) // Revert
CORE_POOL_COMPTRROLLER.setCollateralFactor(...) // Revert

```

Other constructor parameters are checked for zero addresses, but corePoolComptroller_ is not, which leads to inconsistent validation.

Recommendation Add zero address validation for corePoolComptroller_:

JavaScript

```

constructor(

```



```

        ICorePoolComptroller corePoolComptroller_,
        ResilientOracleInterface resilientOracle_,
        OracleInterface sentinelOracle_
    ) {
        if (address(corePoolComptroller_) == address(0)) revert
        ZeroAddress(); //  Add check
        if (address(resilientOracle_) == address(0)) revert ZeroAddress();
        if (address(sentinelOracle_) == address(0)) revert ZeroAddress();

        CORE_POOL_COMPROLLER = corePoolComptroller_;
        RESILIENT_ORACLE = resilientOracle_;
        SENTINEL_ORACLE = sentinelOracle_;
        // ...
    }

```

Status	Not fixed.
--------	------------

[I02] Inconsistent Validation Pattern Between Core Pool and Isolated Pool Logic

Contract	contracts/DeviationSentinel/DeviationSentinel.sol
----------	---

Severity Level	Informational
----------------	---------------

Description	The contract exhibits inconsistent validation patterns between Core Pool and Isolated Pool code paths in the collateral factor management functions.
-------------	--

Issue 1: Missing `isListed` Check in Isolated Pool Restoration.

In `_setCollateralFactorToZero()`, both branches properly verify `isListed` status before modifying collateral factors. However, in `_restoreCollateralFactor()`, the Isolated Pool branch **lacks** this validation.

Issue 2: Inconsistent Return Value Handling.

The Core Pool logic validates `setCollateralFactor()` return values and reverts on failure, while the Isolated Pool path ignores return values in **both functions**.

JavaScript

```

/// @notice Set collateral factor to zero and store original value
/// @param market The market to modify
function _setCollateralFactorToZero(IVToken market) internal {
    IComptroller comptroller = market.comptroller();

```

```

        MarketState storage state = marketStates[address(market)];

        if (address(comptroller) == address(CORE_POOL_COMPTRROLLER)) {
            // Store original CF and LT for each emode group, then set
            to 0
            for (uint96 i = CORE_POOL_COMPTRROLLER.corePoolId(); i <=
CORE_POOL_COMPTRROLLER.lastPoolId(); i++) {
                //...

                if (isListed) {
                    // Store original values for this pool ID
                    state.poolCFs[i] = collateralFactorMantissa;
                    state.poolLTs[i] = liquidationThresholdMantissa;

                    // Set collateral factor to 0, keep liquidation
                    threshold unchanged
                    uint256 result =
CORE_POOL_COMPTRROLLER.setCollateralFactor(
                        i,
                        address(market),
                        0,
                        liquidationThresholdMantissa
                    );
                    if (result != 0) revert ComptrollerError(result);

                    // Emit event for each pool ID
                    emit CollateralFactorUpdated(address(market), i,
collateralFactorMantissa, 0);
                }
            }
        } else {
            IILComptroller.Market memory marketData =
IILComptroller(address(comptroller)).markets(address(market));
            if (marketData.isListed) {
                state.poolCFs[0] = marketData.collateralFactorMantissa;
                state.poolLTs[0] =
marketData.liquidationThresholdMantissa;

                IILComptroller(address(comptroller)).setCollateralFactor(
                    address(market),
                    0,
                    marketData.liquidationThresholdMantissa
                );
                emit CollateralFactorUpdated(address(market), 0,
marketData.collateralFactorMantissa, 0);
            }
        }
    }
}

```

JavaScript

```
/// @notice Restore original collateral factor
/// @param market The market to restore
function _restoreCollateralFactor(IVToken market) internal {
    IComptroller comptroller = market.comptroller();
    MarketState storage state = marketStates[address(market)];

    // Check if this is a core pool or isolated pool
    if (address(comptroller) == address(CORE_POOL_COMPROLLER)) {
        // Core pool - restore original CF and LT for each emode
        group
            for (uint96 i = CORE_POOL_COMPROLLER.corePoolId(); i <=
CORE_POOL_COMPROLLER.lastPoolId(); i++) {
                (bool isListed, , , , , ) =
CORE_POOL_COMPROLLER.poolMarkets(i, address(market));
                if (isListed) {
                    uint256 result =
CORE_POOL_COMPROLLER.setCollateralFactor(
                        i,
                        address(market),
                        state.poolCFs[i],
                        state.poolLTs[i]
                    );
                    if (result != 0) revert ComptrollerError(result);
                }
            }
        } else {
            // Isolated pool
            uint256 originalCF = state.poolCFs[0];
            uint256 originalLT = state.poolLTs[0];

            IILComptroller(address(comptroller)).setCollateralFactor(address(marke
t), originalCF, originalLT);
            emit CollateralFactorUpdated(address(market), 0, 0,
originalCF);
            delete state.poolCFs[0];
            delete state.poolLTs[0];
        }
    }
}
```

Recommendation To ensure consistent and safe behavior across Core Pools and Isolated Pools, the following mitigations are recommended:

- During collateral factor restoration, the Isolated Pool branch should explicitly verify that the market is still listed (`isListed == true`) before restoring cached CF/LT values. This brings the behavior in line with the Core Pool logic and prevents restoring parameters to markets that have been unlisted, re-listed, or otherwise modified through governance during Sentinel intervention.
-

- The return value of `setCollateralFactor` should be checked in the Isolated Pool path, similar to the Core Pool implementation. Any non-zero return value should be treated as a failure and reverted to avoid silent misconfigurations.

Status

Partially fixed in

<https://github.com/VenusProtocol/venus-periphery/pull/44/commits/304beb519bd6e5196c63d83169d6e5af73989c72>

Issue 1 has been fixed; Issue 2 has not been fixed.

[I03] Inconsistent Event Emissions for Isolated Pool Collateral Factor Updates**Contract**

contracts/DeviationSentinel/DeviationSentinel.sol

Severity Level

Informational

Description

The contract defines two overloaded `CollateralFactorUpdated` events with different signatures - one for core pools (with `poolId`) and one for isolated pools (without `poolId`).

However, the isolated pool branches in both `_setCollateralFactorToZero` and `_restoreCollateralFactor` incorrectly emit the 4-parameter core pool event instead of the 3-parameter isolated pool event:

JavaScript

```
/// @notice Emitted when collateral factor is updated for core pool
/// @param market The market address
/// @param poolId The pool ID (emode group)
/// @param oldCF The old collateral factor
/// @param newCF The new collateral factor
event CollateralFactorUpdated(address indexed market, uint96
indexed poolId, uint256 oldCF, uint256 newCF);

/// @notice Emitted when collateral factor is updated for isolated
pool
/// @param market The market address
/// @param oldCF The old collateral factor
/// @param newCF The new collateral factor
event CollateralFactorUpdated(address indexed market, uint256
oldCF, uint256 newCF);

/// @notice Set collateral factor to zero and store original value
/// @param market The market to modify
function _setCollateralFactorToZero(IVToken market) internal {
    IComptroller comptroller = market.comptroller();
    //...
```

```

    } else {
        IILComptroller.Market memory marketData =
        IILComptroller(address(comptroller)).markets(address(market));
        if (marketData.isListed) {
            state.poolCFs[0] =
            marketData.collateralFactorMantissa;
            state.poolLTs[0] =
            marketData.liquidationThresholdMantissa;

            IILComptroller(address(comptroller)).setCollateralFactor(
                address(market),
                0,
                marketData.liquidationThresholdMantissa
            );
            emit CollateralFactorUpdated(address(market), 0,
            marketData.collateralFactorMantissa, 0);
        }
    }

    /// @notice Restore original collateral factor
    /// @param market The market to restore
    function _restoreCollateralFactor(IVToken market) internal {
    ///...
        } else {
            // Isolated pool
            uint256 originalCF = state.poolCFs[0];
            uint256 originalLT = state.poolLTs[0];

            IILComptroller(address(comptroller)).setCollateralFactor(address(marke
            t), originalCF, originalLT);
            emit CollateralFactorUpdated(address(market), 0, 0,
            originalCF);
            delete state.poolCFs[0];
            delete state.poolLTs[0];
        }
    }
}

```

Recommendation Emit the appropriate 3-parameter event for isolated pools in both functions. Or remove the event overloading and use a single, unified event.

Status Not fixed.

[I04] Redundant Code

Contract	contracts/DeviationSentinel/Oracles/PancakeSwapOracle.sol contracts/DeviationSentinel/Oracles/UniswapOracle.sol
Severity Level	Informational
Description	In the PancakeSwapOracle and UniswapOracle contracts, the PriceCalculationError error is declared but never used in the contract logic. As a result, this error definition constitutes redundant code, which may reduce code clarity and increase long-term maintenance overhead. <pre> JavaScript /// @notice Thrown when price calculation fails error PriceCalculationError(); </pre>
Recommendation	It is recommended to remove the PriceCalculationError error definitions declared in the PancakeSwapOracle and UniswapOracle contracts, as they are currently unused.
Status	Fixed in https://github.com/VenusProtocol/venus-periphery/pull/44/commits/cf89978bf8e5a69b6c6af289d4eefefe9489933e

[I05] Token Oracle Configuration Cannot Be Explicitly Removed

Contract	contracts/DeviationSentinel/Oracles/SentinelOracle.sol
Severity Level	Informational
Description	The oracle contract allows privileged roles to configure a price oracle for a token via setTokenOracleConfig. However, there is currently no explicit mechanism to remove or disable an existing oracle configuration once it has been set. <pre> JavaScript function setTokenOracleConfig(address token, address oracle) external { _checkAccessAllowed("setTokenOracleConfig(address,address)"); if (token == address(0)) revert ZeroAddress(); if (oracle == address(0)) revert ZeroAddress(); tokenConfigs[token] = TokenConfig({ oracle: oracle }); emit TokenOracleConfigUpdated(token, oracle); </pre>

```
}
```

Recommendation Two possible solutions:

1. Add a `removeTokenOracleConfig(address token)` function to delete an existing token configuration.
2. Allow the `setTokenOracleConfig` function to accept `address(0)` as a valid oracle value, so the configuration can be explicitly cleared.

Status Not fixed.

Fix Review

Fix Date: 2026/01/21 (YYYY/MM/DD)

Fix Review Date: 2026/01/22 (YYYY/MM/DD)

Overall Risk: **Medium** -> **Informational**

Dev Team's Reply	We've completed the fixes in this PR — https://github.com/VenusProtocol/venus-periphery/pull/44 . Please take a look and let us know if you have any questions or feedback.
Fix PR	https://github.com/VenusProtocol/venus-periphery/pull/44

Fix status

Finding No.	Finding Name	Fix Status
1	[M01] Potential CF/ LT Overwrite When Restoring Core Pool Markets After lastPoolId Increases	Fixed
2	[M02] MMarket CF/LT May Be Incorrectly Modified	Fixed
3	[I01] Constructor Does Not Validate corePoolComptroller_ For Zero	Partially fixed

4	[I02] Inconsistent Validation Pattern Between Core Pool and Isolated Pool Logic	Not fixed
5	[I03] Inconsistent Event Emissions for Isolated Pool Collateral Factor Updates	Not fixed
6	[I04] Redundant Code	Fixed
7	[I05] Token Oracle Configuration Cannot Be Explicitly Removed	Not fixed